CSE 3523

Segment- 8
IoT and Its technology

Contents

01	**Intro to IoT & Infrastructure**	IoT basics, evolution, ecosystem layers, components
02	**Networks & Communication**	Communication models, network types, MQTT/HTTP/CoAP
03	**IoT Processes & Data Mgmt**	Data lifecycle, edge vs cloud,, compression
04	**M2M & IoT Value Chains**	M2M basics, IoT vs M2M comparison, trust, value chain
05	**Architecture, Big Data & Serialization**	3/5-layer architecture, Big Data 5 V
06	**Security, Privacy & Smart Apps**	Security threats, defenses, GDPR, smart cities

Introduction to IoT & Infrastructure

Basics · Evolution · Ecosystem · Architecture Overview

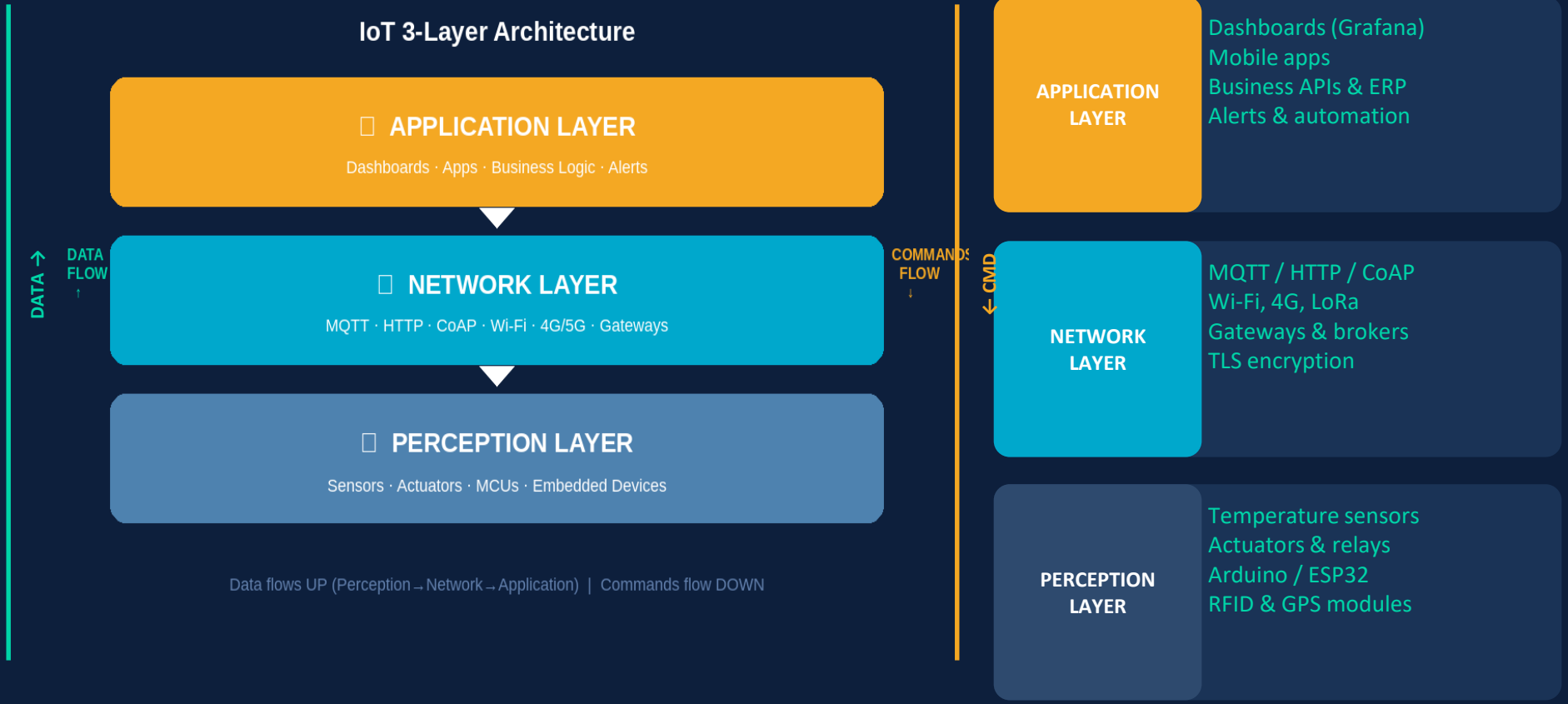
- IoT Definition
- Historical Evolution
- Core Components
- IoT Ecosystem Layers
- Key Applications Preview
- Why IoT Matters

What is the Internet of Things (IoT)?

"A IoT stands for the **Internet of Things**. It refers to a network of things (physical objects) embedded with software and connectivity, allowing them to collect and exchange data over the internet without human intervention

- "Things" = any physical object: a car, a fridge, a streetlight, a hospital bed, an industrial robot — embedded with electronics
- Each device has a unique identifier (IP address or device ID) and communicates over a network
- IoT blurs the boundary between the physical world and digital information systems
- Key enabler: miniaturization of sensors + cheap wireless connectivity + scalable cloud computing
- Estimated 75+ billion connected devices worldwide by 2025 — growing faster than any technology before it

IoT 3-Layer Architecture



IoT 3-Layer Architecture — Definition & Purpose

DEFINITION

What is the 3-Layer Architecture?

The IoT 3-Layer Architecture is a standardised framework that organises an IoT system into three distinct functional layers — Perception, Network, and Application — each with defined responsibilities, enabling modular design, interoperability, and scalability.

LAYER 1: PERCEPTION LAYER

What it does: Senses the physical world and generates raw data

- *Output: Analog or digital signals converted to data (e.g., 23.5°C, door=OPEN, lat=23.81)*

LAYER 2: NETWORK LAYER

What it does: Transports data from devices to cloud and delivers commands back

- *Protocols handled: MQTT, HTTP, CoAP, Zigbee, Z-Wave, LoRaWAN, 4G/5G*

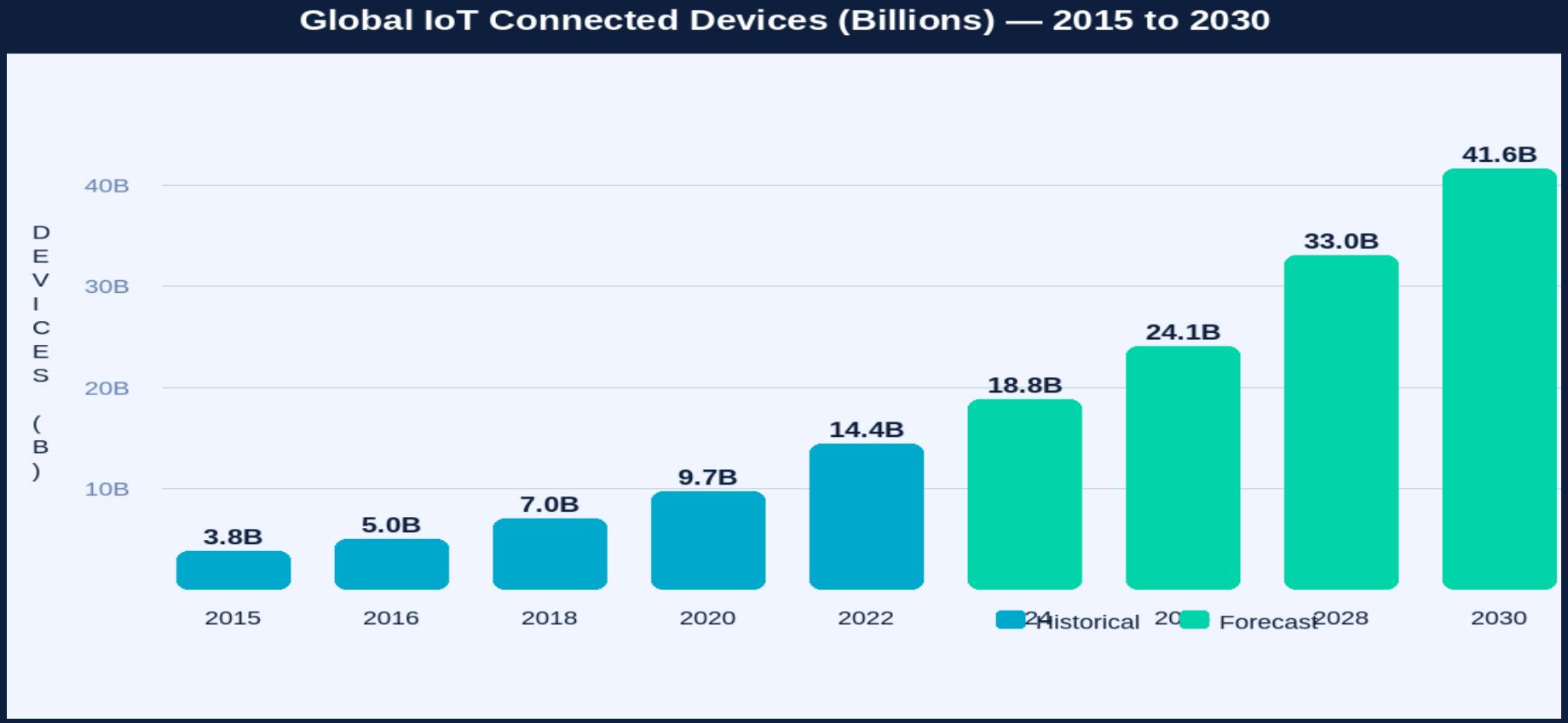
LAYER 3: APPLICATION LAYER

What it does: Processes data and delivers value to end users and business systems

- *Services: Storage, analytics, visualisation, automation rules, remote device control*

★ Data flows UPWARD (Perception → Network → Application) | Commands flow DOWNWARD (Application → Network → Perception)

Global IoT Device Growth — 2015 to 2030



The Four Pillars That Define IoT

1 Sensing

Physical world data captured by sensors:
- Temperature, humidity, pressure
- Motion, proximity, light
- Biometric (heartrate, glucose)
- Chemical, acoustic, vibration

Actuators convert digital signals BACK to physical actions (motors, relays, pumps).

2 Connectivity

Every device must communicate:
- Short range: Bluetooth, NFC, Zigbee
- Medium range: Wi-Fi, Z-Wave
- Long range: LoRaWAN, Sigfox, NB-IoT
- Wide area: 4G LTE, 5G cellular

Protocol choice depends on power, range, and data rate needs.

3 Data Processing

Raw sensor data → Actionable insights:
- Edge: On-device or local gateway
- Fog: Intermediate layer near network edge
- Cloud: Centralized, scalable servers

Includes filtering, aggregation, ML inference, anomaly detection, and storage.

4 Action & Automation

Closed-loop feedback based on data:
- Automated responses (HVAC adjusts)
- Alerts & notifications to humans
- Dashboards and visualizations
- Integration with business systems

Goal: minimal human intervention needed.

IoT vs Traditional Networked Systems

Traditional Networked Systems

- Human-initiated: Actions require user commands to execute
- Periodic monitoring: Data collected manually or at scheduled intervals
- Limited scale: Hundreds to thousands of endpoints at most
- High latency: Decisions require human review before action
- Purpose-built: Each system designed for one specific task
- Siloed data: Systems don't share data with each other
- High operational cost: Requires technicians for monitoring

IoT Systems

- Autonomous: Devices sense, decide, and act without humans
- Continuous monitoring: Real-time 24/7 data collection
- Massive scale: Billions of devices on single platforms
- Low latency: Edge processing enables millisecond responses
- Interoperable: Devices from different vendors can integrate
- Data sharing: Open APIs enable ecosystem-wide analytics
- Cost-efficient: Automated alerts reduce operational workforce

IoT Hardware Components

Sensors

Convert physical quantities to electrical signals.
Types: Thermistor (temp), PIR (motion), LDR (light), DHT22 (humidity), GPS module, accelerometer, gyroscope, CO2, pH sensor.
Output: analog OR digital (I2C, SPI, UART).

Actuators

Convert digital commands to physical actions.
Types: DC/servo motors, solenoid valves, relays, LED strips, buzzers, LCD displays.
Used in: industrial robots, smart locks, irrigation systems.

Microcontrollers (MCU)

Low-power, single-chip computers for embedded logic.
Examples: Arduino Uno (ATmega328, 32KB flash), ESP8266 (Wi-Fi built-in), STM32 series.
Used when: simple logic + low power needed.

Single-Board Computers

Full Linux computer on one board.
Examples: Raspberry Pi 4 (4-core ARM, 4GB RAM), NVIDIA Jetson Nano (AI inference).
Used when: complex processing, ML, video analysis needed.

IoT Gateways

Bridge local devices to cloud. Handle protocol translation (Zigbee → MQTT), local buffering, security screening.
Examples: AWS Greengrass, Cisco IR829, Raspberry Pi as gateway.

Connectivity Modules

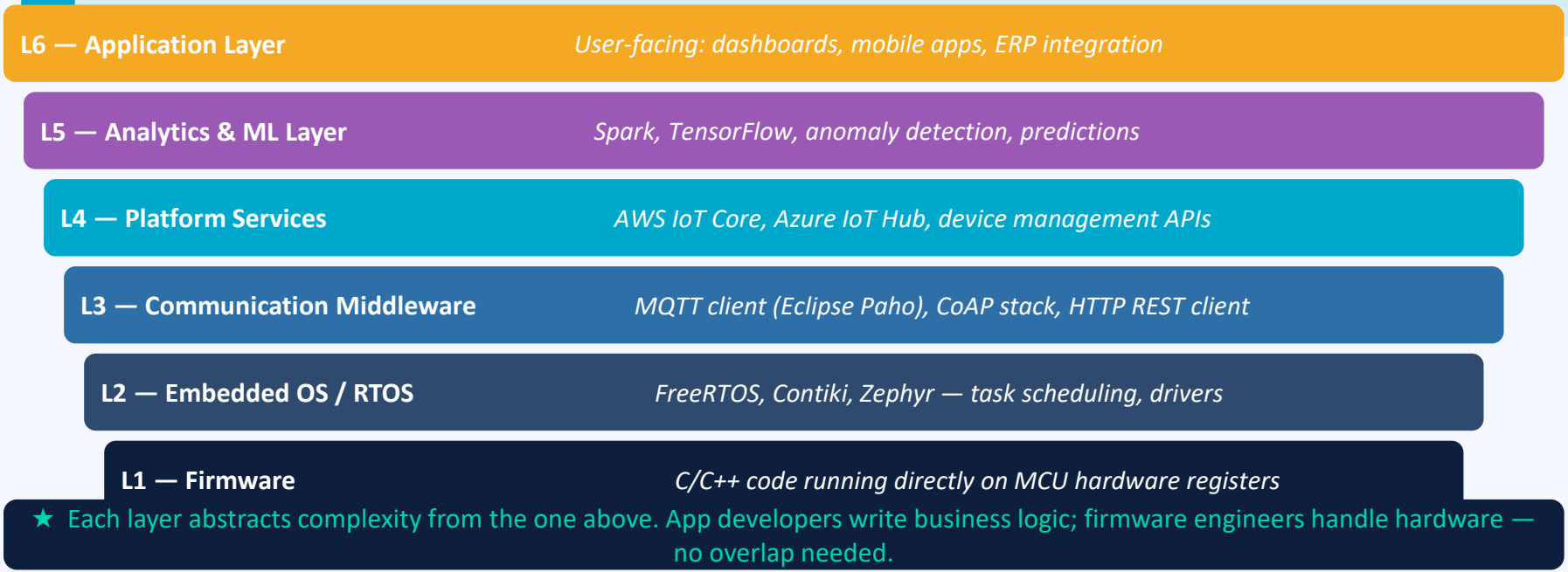
Add wireless to MCUs.
Examples: ESP8266 (Wi-Fi), HC-05 (Bluetooth), SIM800L (GSM/GPRS), LoRa SX1276.
Selected based on range, power, bandwidth needs.

IoT Software Stack — Definition & Why It Matters

DEFINITION

What is the IoT Software Stack?

The IoT Software Stack is the complete set of software layers running on IoT devices and cloud infrastructure — from the lowest-level firmware that talks directly to hardware, up to the cloud applications that deliver value to users. Each layer abstracts complexity from the layer above it.



IoT Software Stack — From Firmware to Applications

- Firmware (Level 1): Low-level code running directly on hardware. Written in C/C++ for MCUs. Handles hardware abstraction, sensor read cycles, interrupt routines. Example: Arduino sketches, FreeRTOS tasks.
- Embedded OS (Level 2): Real-time operating systems for resource-constrained devices. Examples: FreeRTOS, Contiki, RIOT OS, Zephyr. Provide task scheduling, memory management, hardware drivers.
- Communication Middleware (Level 3): Software managing data exchange between device and cloud. MQTT client libraries (Eclipse Paho), CoAP stacks, HTTP REST clients. Handles connection retries and QoS.
- IoT Platform Services (Level 4): Cloud-side software for device management, data ingestion, rule engines, and storage. Examples: AWS IoT Core (rules engine + Lambda), Azure IoT Hub (device twins).
- Analytics & ML Layer (Level 5): Processing raw IoT data into insights. Apache Spark for batch, Apache Flink for streaming. TensorFlow Lite / ONNX for edge ML inference. Anomaly detection, predictive models.
- Application Layer (Level 6): User-facing interfaces — web dashboards (Grafana, Node-RED), mobile apps, ERP/CRM integration. REST APIs expose IoT data to business systems.

IoT Ecosystem — How All Layers Connect End to End

DEFINITION

The Complete IoT Ecosystem Flow:

An IoT ecosystem is the complete system of physical devices, communication networks, software platforms, and human interfaces that work together to sense the physical world, transmit data, process it into insights, and take automated or human-driven actions.

Physical World	Sensor Layer	Edge / Gateway	Cloud Platform	Analytics / AI	Action / UI
Events occur	*Perception layer Detects & measures*	*Network layer Transmit & filter*	*Process & store*	*Analyse & predict*	*Automate & alert*
Temp rises in factory room	*DHT22 sensor reads 38°C*	*Gateway filters, sends to MQTT broker*	*AWS IoT stores in InfluxDB*	*ML detects overheating trend*	*Triggers HVAC + SMS alert*

The three pillars of a successful IoT ecosystem:

1 Connectivity Everywhere

Every device must be reachable. The right protocol (MQTT/CoAP/HTTP) and the right network (Wi-Fi/LoRa/NB-IoT) ensure no device is an island.

2 Data Must Flow Reliably

A missed reading, a corrupted packet, or a clock-skewed timestamp can corrupt an entire analysis. QoS, TLS, NTP sync, and validation pipelines are non-negotiable.

3 Action Must Close the Loop

Data with no consequence creates no value. Automation, alerts, and feedback must translate insight into real-world change — the economic justification for all the complexity above.

Why IoT Matters — Business & Social Impact

Operational Efficiency

20–30% reduction in operational costs through automation and predictive maintenance in manufacturing and logistics sectors.

Energy Savings

15–40% reduction in energy consumption via smart building systems that optimize HVAC, lighting, and equipment usage dynamically.

Healthcare Outcomes

40% reduction in hospital readmissions through remote patient monitoring (RPM) and early intervention via wearable health sensors.

Traffic Efficiency

25% reduction in urban traffic congestion using adaptive smart traffic management systems with real-time sensor data.

Downtime Reduction

70% reduction in unplanned equipment downtime using IoT-based predictive maintenance in industrial environments.

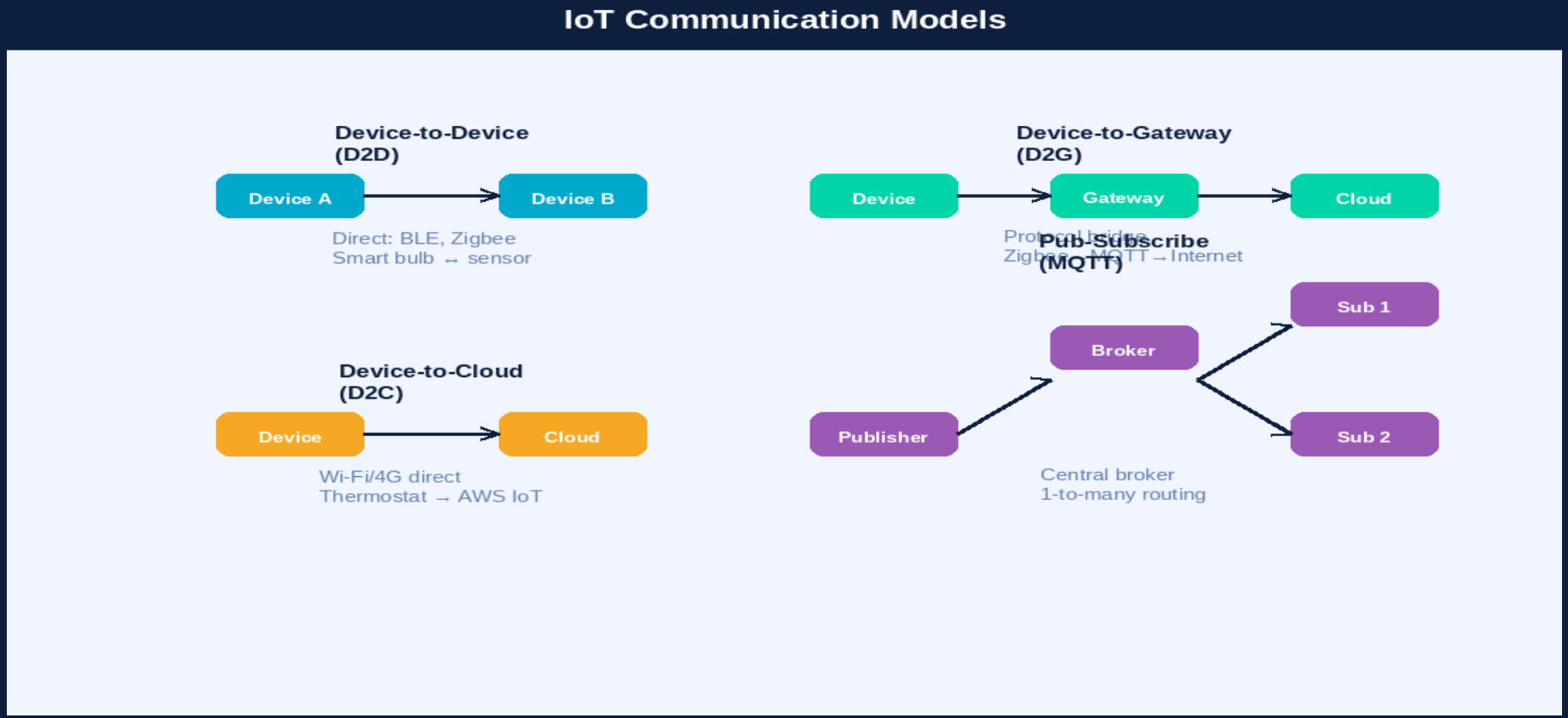
Revenue Generation

$12.6 trillion global economic impact of IoT by 2030 through new business models, service monetization, and efficiency gains.

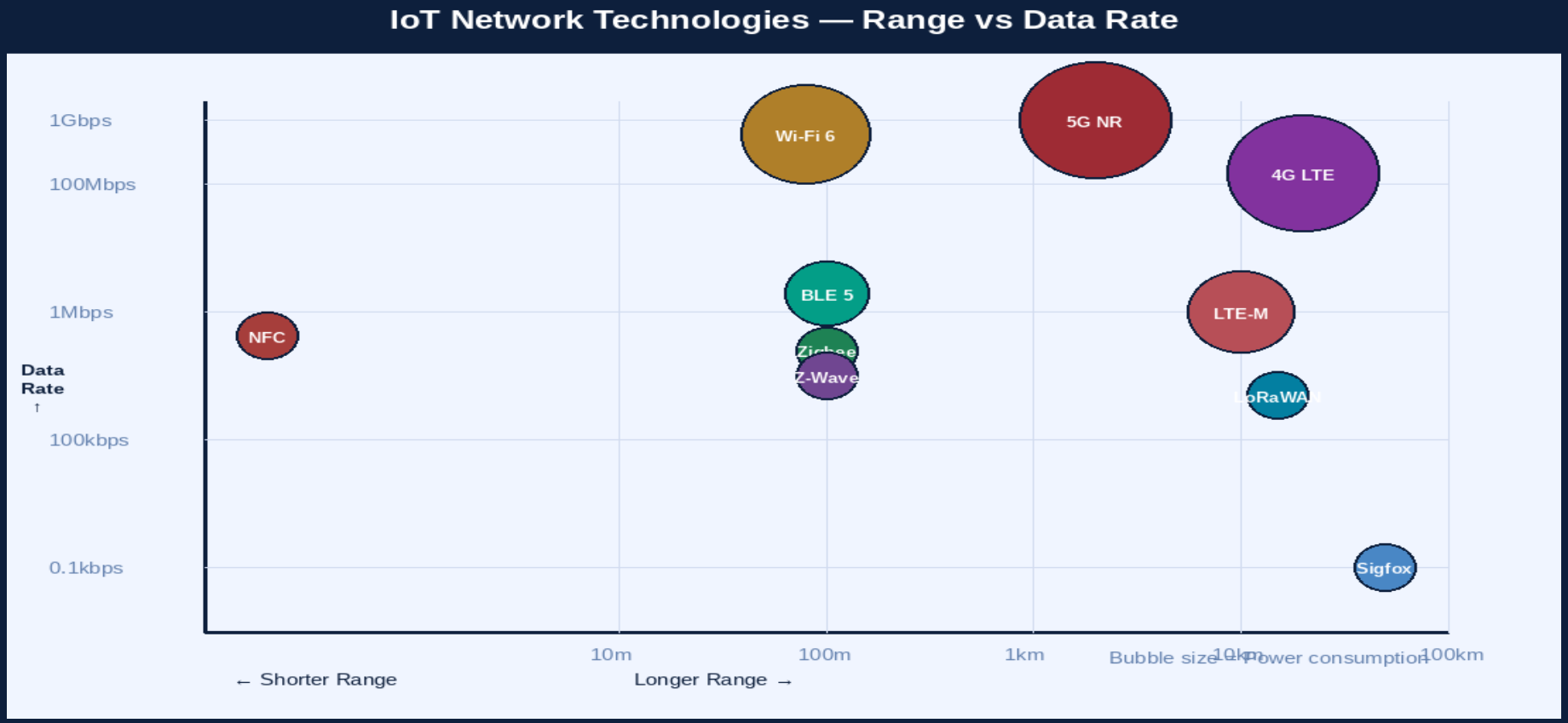
Networks & Communication in IoT

Models · Network Types · Protocols · Data Flow

IoT Communication Models —



IoT Network Technologies — Range vs Data Rate



What is a Communication Protocol?

DEFINITION

Communication Protocol — Definition

A communication protocol is a set of rules and standards that define how data is formatted, transmitted, received, and acknowledged between two or more devices. Without agreed-upon protocols, devices from different manufacturers cannot understand each other.

Real-World Analogy:

A protocol is like a language. Two people can only understand each other if they speak the SAME language and follow the same grammar rules. Similarly, two IoT devices can only exchange data if they use the same protocol with the same message format and rules.

Why Protocols Are Essential in IoT:

Interoperability
Devices from different vendors (Samsung sensor, Arduino, AWS cloud) can communicate because they all speak MQTT.

Efficiency
Protocols define the minimum bytes needed to convey a message — critical for battery-powered sensors where every bit costs energy.

Reliability
Protocols specify how to handle lost messages, retransmit data, and confirm delivery — ensuring data integrity over unreliable networks.

Scalability
Standard protocols allow millions of devices to connect to the same broker/server without each needing custom integration code.

★ In IoT: choosing the wrong protocol = wasted battery + missed data + incompatible systems. Protocol choice is an architectural decision.

Types of IoT Communication Protocols

IoT protocols are classified by their purpose and operating layer:

Application Layer Protocols

MQTT
Lightweight pub-sub for IoT telemetry streaming. TCP-based, 2-byte header. Most popular for IoT.

HTTP / REST
Request-response web protocol. Familiar, widely supported. Heavy for constrained devices.

CoAP
Constrained Application Protocol. UDP-based REST for low-power sensor networks (LLNs).

AMQP
Advanced Message Queuing. Enterprise-grade messaging. Used in industrial IoT platforms.

Network / Connectivity Protocols

Wi-Fi (802.11)
High bandwidth, short range, medium power. Smart home, office IoT.

Bluetooth / BLE
Short range (<100m), very low power. Wearables, beacons, health sensors.

Zigbee / Z-Wave
Mesh network, ultra-low power. Smart lighting, building automation.

LoRaWAN
Long range (15km), ultra-low power. Agriculture, smart cities, utilities.

NB-IoT / LTE-M
Cellular LPWAN. Nationwide coverage via telcos. Smart meters, tracking.

Data / Serialization Formats

JSON
JavaScript Object Notation. Human-readable, self-describing. Most popular for IoT payloads.

XML
eXtensible Markup Language. Verbose but schema-validated. Legacy/enterprise systems.

CBOR
Concise Binary Object Representation. Binary JSON. 50% smaller. Constrained devices.

Protobuf
Google Protocol Buffers. Binary, typed, very compact. High-frequency industrial IoT.

MQTT — Message Queue Telemetry Transport: Complete Definition

DEFINITION

What is MQTT?

MQTT (Message Queue Telemetry Transport) is a lightweight, open-standard, publish-subscribe messaging protocol designed for constrained IoT devices and unreliable networks. Created by IBM in 1999 for satellite-linked oil pipelines, it is now the de-facto standard for IoT telemetry.

Key Technical Facts

- Standard: ISO/IEC 20922 — open OASIS standard since 2014
- Transport: TCP/IP — port 1883 (plain), 8883 (TLS encrypted)
- Header: Only 2 bytes fixed header — minimal overhead
- Pattern: Publish-Subscribe through a central MQTT Broker
- Topics: Hierarchical strings e.g. home/bedroom/temperature
- Wildcards: + (single level), # (multi-level) in subscriptions
- Retained messages: Broker caches last message for new subscribers
- Will message: Device pre-registers a 'last will' sent if it disconnects
- Brokers: Mosquitto, HiveMQ, AWS IoT Core, Azure IoT Hub

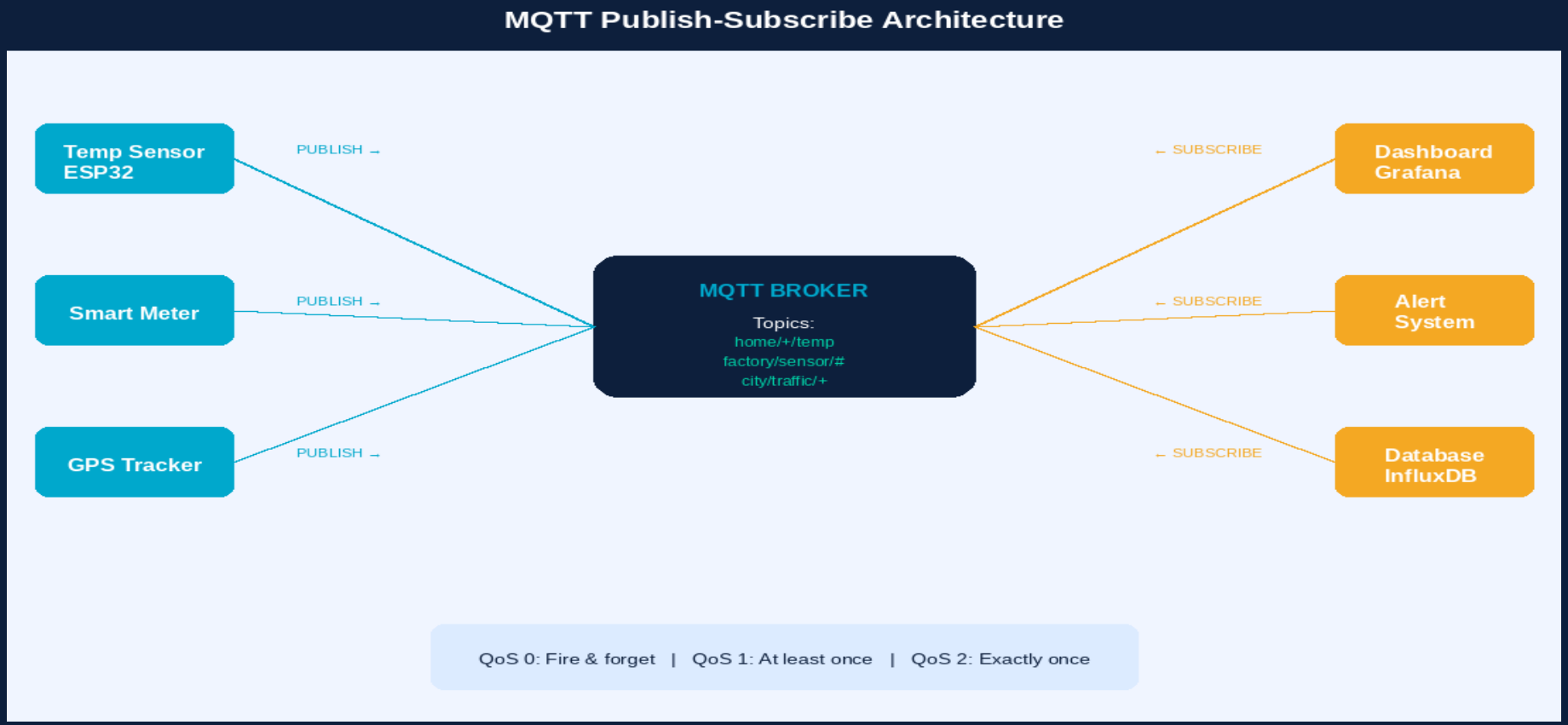
✓ Use MQTT When...

- ✓ Continuous sensor data streaming to cloud
- ✓ Many devices publishing to same broker (many-to-one)
- ✓ Battery-powered devices with limited bandwidth
- ✓ Unreliable network — need guaranteed delivery (QoS

✗ Avoid MQTT When...

- ✗ Complex request-response interactions needed
- ✗ Direct browser access (use WebSocket bridge instead)
- ✗ Large file transfers (use HTTP/FTP instead)

MQTT Publish-Subscribe Architecture —



MQTT Protocol — How It Works Step by Step



Quality of Service (QoS) Levels

QoS 0 — At Most Once

Non-critical readings
(ambient temperature, humidity)
One delivery attempt; no ACK; may be lost.

QoS 1 — At Least Once

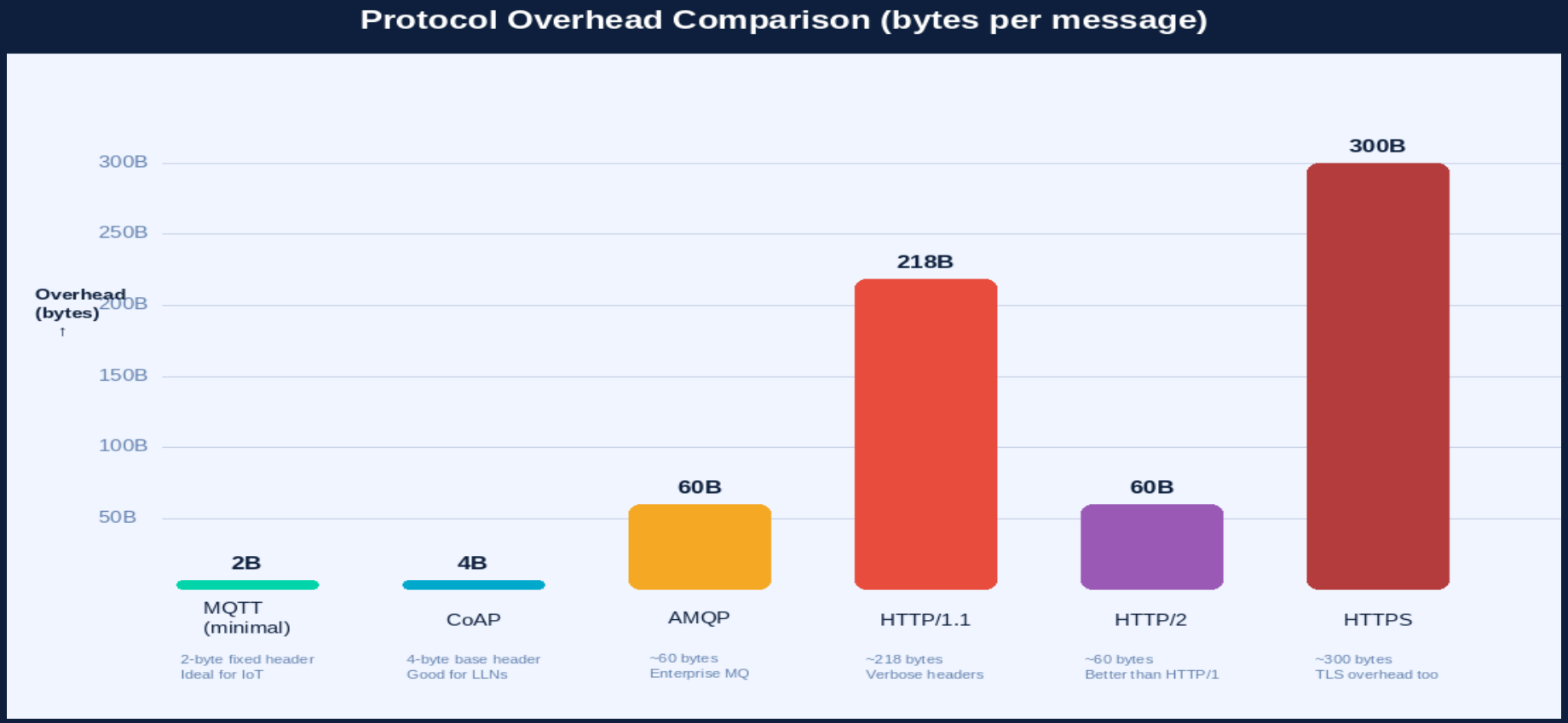
Alerts & notifications
ACK required; sender retransmits until confirmed.
Duplicates possible — receiver must handle.

QoS 2 — Exactly Once

Billing, medical dosing, payments
4-step handshake ensures exactly one delivery.
Slowest but most reliable.

Port 1883 (plain) · Port 8883 (TLS encrypted) · Topic wildcards: + (single level) #(multi-level) · Header: only 2 bytes · Broker examples: Mosquitto, HiveMQ, AWS IoT Core

Protocol Overhead Comparison — Bytes per Message



HTTP and CoAP — Definitions & Role in IoT

HTTP — HyperText Transfer Protocol

Definition: HTTP is the foundational request-response protocol of the World Wide Web. In IoT it is used when devices need to communicate with REST APIs, web services, or cloud platforms using standard web infrastructure. HTTP 1.1 sends a full request and waits for a full response before sending the next — making it reliable but bandwidth-heavy.

Transport:	TCP — port 80 (HTTP), 443 (HTTPS)	**Header overhead:**	~218–300 bytes per request (very verbose)
Pattern:	Request-Response — client asks, server answers	**Security:**	HTTPS with TLS — mature, widely understood
Best for IoT:	REST APIs, cloud portal communication, infrequent large data uploads	**Not good for:**	High-frequency telemetry, battery-powered devices, constrained networks

CoAP — Constrained Application Protocol

Definition: CoAP is a specialised web protocol designed for resource-constrained devices (microcontrollers with limited RAM/CPU) on Low-power Lossy Networks (LLNs). It is the UDP-based, binary equivalent of HTTP — same REST principles (GET/PUT/POST/DELETE) but with a fraction of the overhead. Defined in IETF RFC 7252.

Transport:	UDP — port 5683 (plain), 5684 (DTLS secured)	**Header overhead:**	Only 4 bytes base header — 50× smaller than HTTP

MQTT vs HTTP vs CoAP — Detailed Visual Comparison

MQTT		HTTP / REST		CoAP	
Pattern:	Publish-Subscribe (via broker)	**Pattern:**	Request-Response (client-server)	**Pattern:**	Request-Response (REST-like)
Transport:	TCP — Port 1883 / 8883 (TLS)	**Transport:**	TCP — Port 80 / 443 (HTTPS)	**Transport:**	UDP — Port 5683
Header:	2 bytes fixed header (minimal!)	**Header:**	~218 bytes overhead per request	**Header:**	4 bytes base header (binary)
QoS:	3 built-in levels (0,1,2)	**QoS:**	None built-in (TCP reliability only)	**QoS:**	4 levels (CON/NON/ACK/RST)
Broker?:	Yes — central routing broker	**Broker?:**	No — direct client ↔ server	**Broker?:**	No — direct peer-to-peer
Security:	TLS/SSL (MQTTS)	**Security:**	HTTPS (TLS)	**Security:**	DTLS (Datagram TLS)
					Constrained LLN sensor

MQTT vs HTTP vs CoAP — Which Protocol Should You Choose?

Use the right protocol for the right scenario:

Scenario

Temperature sensor sends data every 30 sec to cloud

→ Use: MQTT

Why: Lightweight, pub-sub, no need to wait for response. Battery-friendly. QoS 0 is sufficient.

Scenario

Motion detector must guarantee alarm reaches server

→ Use: MQTT QoS 2

Why: Exactly-once delivery guarantee via 4-step handshake. Use when action has real-world consequences.

Scenario

Mobile app fetches current device status on demand

→ Use: HTTP / REST

Why: Request-response perfectly fits 'fetch current state' pattern. Familiar to web developers.

Scenario

Sensor on coin-cell battery in Zigbee mesh network

→ Use: CoAP

Why: 4-byte UDP header, runs on 6LoWPAN. Minimal power consumption. REST-like simplicity.

Scenario

Dashboard streams live data to 1,000 subscribers

→ Use: MQTT

Why: Broker fans out one published message to all 1,000 subscribers — O(1) publisher effort.

Scenario

SCADA system integrates with legacy enterprise API

→ Use: HTTP / SOAP

Why: Enterprise systems almost always expose HTTP REST or SOAP endpoints. Use what exists.

Complete IoT Data Transmission Flow

- Step 1 — Sensor Reading: Analog/digital signal generated by physical sensor. ADC converts analog voltage to digital value. Firmware applies calibration formula (e.g., raw_ADC × 0.48 − 50 = temperature in °C).
- Step 2 — Edge Processing: Gateway or MCU filters noise, applies threshold checks, aggregates multiple readings. Only meaningful data forwarded to cloud — reduces bandwidth 60–90%.
- Step 3 — Protocol Encoding: Data serialized into JSON or binary MQTT payload. TLS handshake establishes encrypted channel. MQTT CONNECT → CONNACK → PUBLISH → PUBACK sequence.
- Step 4 — Internet Transit: Encrypted packet routed via internet backbone. IoT broker (Mosquitto, HiveMQ, AWS IoT) receives published message. Subscribers notified via PUSH (no polling).
- Step 5 — Cloud Processing: Rules engine evaluates incoming data. Triggers Lambda functions, inserts into InfluxDB, streams to Kafka topic. ML model scores for anomaly detection in real-time.
- Step 6 — Action & Visualization: Grafana dashboard updates. Alert sent via PagerDuty if threshold exceeded. Command published back to device topic → actuator responds within 200–500ms.

IoT Network Security — Transport Layer

Common Threats

- Eavesdropping: Passive interception of unencrypted Wi-Fi/BLE packets using Wireshark, Kismet
- MitM Attack: Attacker inserts between device and broker, reads or modifies data in transit
- Replay Attack: Captured valid packets re-sent later to trigger unauthorized actions
- RF Jamming: Flooding wireless channel with interference — denial of service
- Rogue Gateway: Fake base station captures device credentials
- Packet Injection: Malicious packets injected into unprotected channels

Transport Security Solutions

- TLS 1.3: Encrypt all MQTT/HTTP traffic. Mutual TLS (mTLS) authenticates both device AND server
- DTLS: Datagram TLS for UDP-based CoAP — same security as TLS over connectionless transport
- Certificate Pinning: Device only accepts server certificates matching pre-stored fingerprint
- VPN Tunnels: Site-to-site VPNs for industrial IoT. WireGuard is lightweight for IoT gateways
- Network Segmentation: IoT devices on isolated VLANs — compromise of one device doesn't spread
- Intrusion Detection: Anomaly-based IDS monitors traffic patterns for unusual device behaviour

IoT Processes & Data Management

Data Lifecycle · Processing · Edge vs Cloud · Aggregation

- IoT Data Lifecycle
- Data Types in IoT
- Edge Computing
- Cloud Processing
- Aggregation Techniques
- Storage Systems

IoT Data Lifecycle — Definition & Importance

DEFINITION

What is the IoT Data Lifecycle?

The IoT Data Lifecycle is the complete journey that data takes from the moment a sensor generates a raw reading to the moment it triggers a meaningful action or business decision. Every IoT system must design all six stages — ignoring any stage leads to wasted data, poor decisions, or system failure.

Why Every Stage Matters:

01 Generation

If skipped: No data exists. The entire IoT system is blind.

Design: Choose correct sensor type, sampling rate, data format.

02 Collection

If skipped: Data generated but never reaches any system.

Design: Push vs pull, batch vs streaming, gateway aggregation.

03 Processing

If skipped: Raw, noisy, uncalibrated data enters storage and corrupts analytics.

Design: Filter at edge, normalise units, detect anomalies early.

04 Storage

If skipped: Insights cannot be derived; data lost permanently after analysis.

Design: Time-series DB for recent, data lake for archive, tiered retention.

05 Analysis

If skipped: Data sits unused — common! Most IoT data collected is never analysed.

Design: Descriptive → Predictive → Prescriptive analytics hierarchy.

06 Action

If skipped: All the above effort generates no real-world value or change.

Design: Automated vs human-triggered; feedback loop closes the cycle.

IoT Data Lifecycle — 6 Stages with Details



01 Generation

Sensors produce analog/digital signals continuously. Types: numeric, boolean, GPS, image, audio. Rate: 1 per hour (meter) to 10,000 per second (vibration).

02 Collection

Push (device sends on schedule) or Pull (server polls). Gateways batch from many local devices before cloud upload.

03 Processing

Filter noise, normalise units, enrich with metadata, detect anomalies. Can happen at edge (<10ms) or cloud (scalable).

04 Storage

InfluxDB (time-series), S3 data lake (raw archive). Hot→Warm→Cold tiering. Retention policies: 7d / 90d / 7yr.

05 Analysis

Descriptive (what happened?), Predictive (what will?), Prescriptive (what to do?). Tools: Spark, Grafana, Python/sklearn.

06 Action

Automated command to actuator, push alert to engineer, dashboard update. Feedback loop triggers new sensing.

Edge vs Cloud Processing — Definitions

⚡ Edge Computing — Definition

Processing data at or very near the source — on the IoT device itself, a local gateway, or a nearby edge server — rather than sending raw data to a distant cloud data centre. The goal is to reduce latency, save bandwidth, and enable offline operation.

☁ Cloud Computing — Definition

Processing data in large centralised data centres accessed via the internet. Cloud platforms provide virtually unlimited compute, storage, and analytics capabilities. Raw data travels from IoT devices across the internet to cloud servers for processing.

The 3 Levels of Edge Processing:

1

Mist Computing (On-Device)

Processing runs directly ON the microcontroller or sensor chip. Example: TensorFlow Lite Micro detecting a wakeword ('Alexa') on a tiny chip — never sending audio to cloud unless wakeword detected. Latency: <1ms.

2

Edge Computing (Gateway/Edge Server)

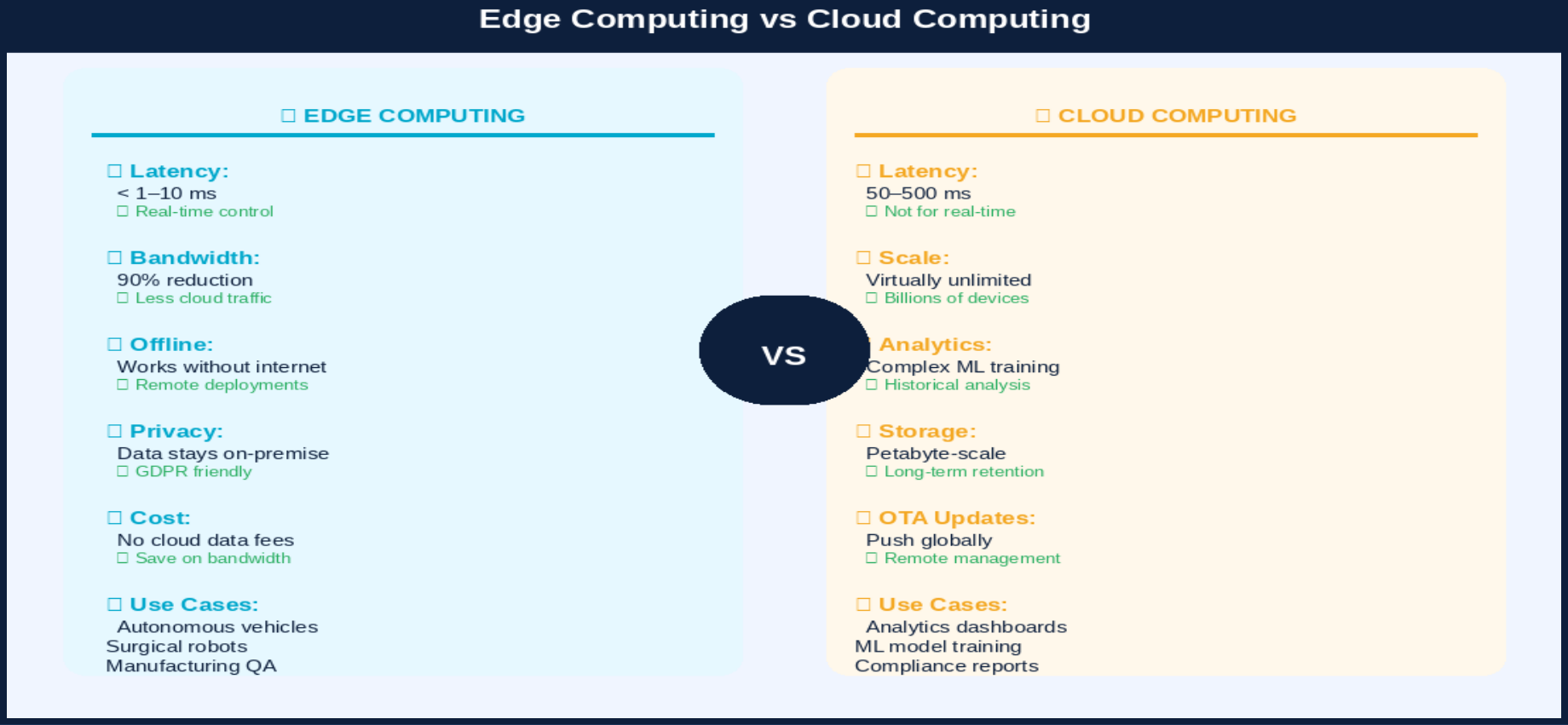
Processing runs on a local gateway or edge server (e.g. Raspberry Pi, Nvidia Jetson Nano). Example: Smart camera running object detection locally, only sending 'person detected' event to cloud. Latency: 5–50ms.

3

Fog Computing (Telecom Edge/MEC)

Processing runs at telecom infrastructure — mini data centres co-located with 5G towers. 5G Multi-access Edge Computing (MEC). Example: Connected vehicles using intersection management with <20ms latency.

Edge vs Cloud Processing



Edge vs Cloud Processing —Comparison

⚡ Edge Processing

- Latency: <1–10 ms response — critical for real-time control, safety systems
- Bandwidth: Process locally, only send summaries — 90%+ bandwidth reduction
- Offline operation: Works without internet — essential for remote deployments
- Data privacy: Sensitive data never leaves premises — GDPR friendly
- Cost: No per-MB cloud data transfer fees — but upfront hardware cost
- Limitations: Limited compute, harder to update software remotely
- Use cases: Autonomous vehicles, industrial control, smart cameras

☁ Cloud Processing

- Latency: 50–500 ms round-trip — acceptable for analytics but not real-time control
- Compute power: Virtually unlimited — run complex ML models, train neural networks
- Storage: Petabyte-scale at low cost — historical analysis over years of data
- Accessibility: Access data from anywhere globally — multi-tenant, multi-region
- OTA updates: Push software updates to devices worldwide simultaneously
- Cost model: Pay-per-use — scales with data volume, no upfront hardware
- Use cases: Analytics dashboards, ML training, compliance reporting

IoT Data Quality — Challenges & Solutions

Common Data Quality Problems

- Missing data: Sensor offline, network outage causes gaps. A temperature sensor might miss 5% of readings during connectivity issues.
- Sensor drift: Sensors lose calibration over time. A pH sensor may read 7.2 when true pH is 7.0 after 6 months deployed.
- Stuck sensor: Reports same value repeatedly (firmware bug or failure). Reading 23.5°C for 24 hours is suspicious.
- Clock skew: Device clock drifts if no NTP sync. Events appear out-of-order — breaks time-series analysis.
- Outliers: Cosmic ray bit-flip causes sensor to report -9999°C. Must detect and remove statistically impossible readings.
- Duplicate messages: MQTT QoS 1 retransmits — same sensor reading appears twice. Must deduplicate before storage.

Data Quality Solutions

- Interpolation: Fill missing gaps using linear or spline interpolation. Mark interpolated values with quality flag.
- Calibration schedules: Regular lab calibration of field sensors. Embedded self-calibration using known reference.
- Anomaly detection: If delta > 3σ from mean, flag as outlier. Z-score, IQR, and isolation forest ML models.
- NTP time sync: All devices sync with NTP server at boot and every hour. PTP for <1µs sync in industrial settings.
- Schema validation: JSON Schema validates incoming messages. Reject messages that don't conform before storage.
- Deduplication: Redis cache with message ID hash (TTL=5 min). ClickHouse ReplacingMergeTree at DB level.

M2M Communication & IoT Value Chains

M2M Basics · IoT vs M2M · Trust · Value Creation

- M2M Definition
- M2M Architecture
- IoT vs M2M Deep Compare
- M2M Trust & Security
- IoT Value Chain
- Business Models

Machine-to-Machine (M2M) Communication — Complete Overview

Definition:

M2M is the direct, automated exchange of data between devices without human intervention, enabling machines to make decisions and trigger actions autonomously.

- Origins: Industrial automation (1970s SCADA systems), then remote monitoring of ATMs, vending machines (1980–90s), then 2G cellular M2M for fleet tracking
- M2M Architecture: End device (sensor/actuator) → Communication network → M2M middleware server → Business application
- Network types: Wired (RS-485, Ethernet, Modbus), cellular (dedicated M2M SIM), satellite (Iridium for remote areas)
- Examples: Smart electricity meters auto-reporting to utility, ATM transactions to bank, GPS tracker to fleet management system

M2M vs IoT — Comprehensive Comparison ⚠

Dimension	M2M	IoT
Communication	Point-to-point, device-to-server	Many-to-many: devices, cloud, users, APIs
Network	Cellular (proprietary SIM), wired	IP-based: internet, Wi-Fi, LPWAN, BLE
Standards	Proprietary, vendor-specific	Open: MQTT, HTTP, CoAP, AMQP, IEEE
Scale	Thousands of devices per deployment	Billions of devices globally
Intelligence	Limited: predefined rules only	AI/ML, edge inference, adaptive behaviour
Integration	Closed, vertical silo	Open APIs, ecosystem, cloud marketplace
User Interface	None or minimal web portal	Rich apps, voice, dashboards, AR
Device Identity	SIM-based (IMEI/IMSI)	PKI certificates, cloud device IDs
Update Mechanism	Manual, on-site firmware update	OTA (Over-The-Air) remote updates

★ *M2M is a subset/predecessor of IoT. IoT extends M2M with internet-scale, open standards, and rich intelligence.*

Trust in M2M Systems — Security Architecture

- Device Identity: Every M2M device must have unique, verifiable identity. IMEI (hardware ID) + IMSI (SIM ID) for cellular. PKI X.509 certificates for IP-based M2M. Hardware Security Modules (HSM) store private keys securely.
- Secure Enrollment: Process of registering a new device with the M2M platform. Zero-touch provisioning: certificate burned in at factory (EST — Enrollment over Secure Transport). Prevents unauthorized devices joining network.
- Authentication: Device proves identity before data exchange. Pre-shared key (PSK) — simple but complex at scale. Certificate-based mutual TLS (mTLS) — server AND device authenticate each other. OAuth 2.0 device flow.
- Authorization (RBAC): Authenticated device can only access its own data. A temperature sensor can only publish to /sensors/temp/{id} — cannot read other devices. Policy enforced at broker/API gateway.
- Data Integrity: MAC (Message Authentication Code) ensures message not tampered with. HMAC-SHA256 appended to M2M payload. Receiver verifies before processing. Prevents replay and injection attacks.
- Trust Anchor Hardware: TPM (Trusted Platform Module) chips store root of trust. ARM TrustZone creates secure execution environment on MCU. Secure boot verifies firmware integrity before execution.

IoT Value Chain — Definition & Business Significance

DEFINITION

What is the IoT Value Chain?

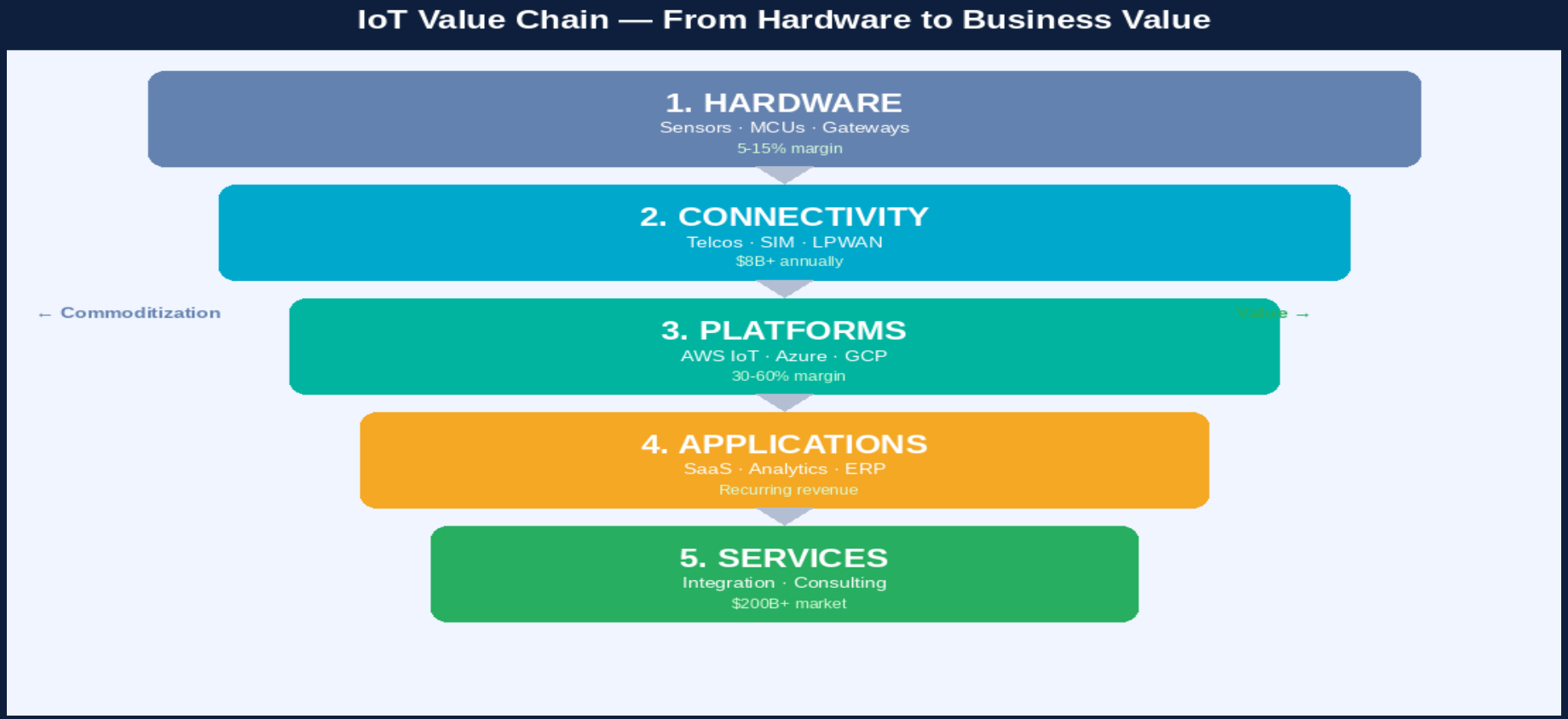
The IoT Value Chain describes the sequence of stages through which raw physical hardware is transformed into business value and customer outcomes. Each stage adds a layer of capability and commands higher profit margins, with value concentrating toward the application and services end of the chain.

The 5 Stages Explained:

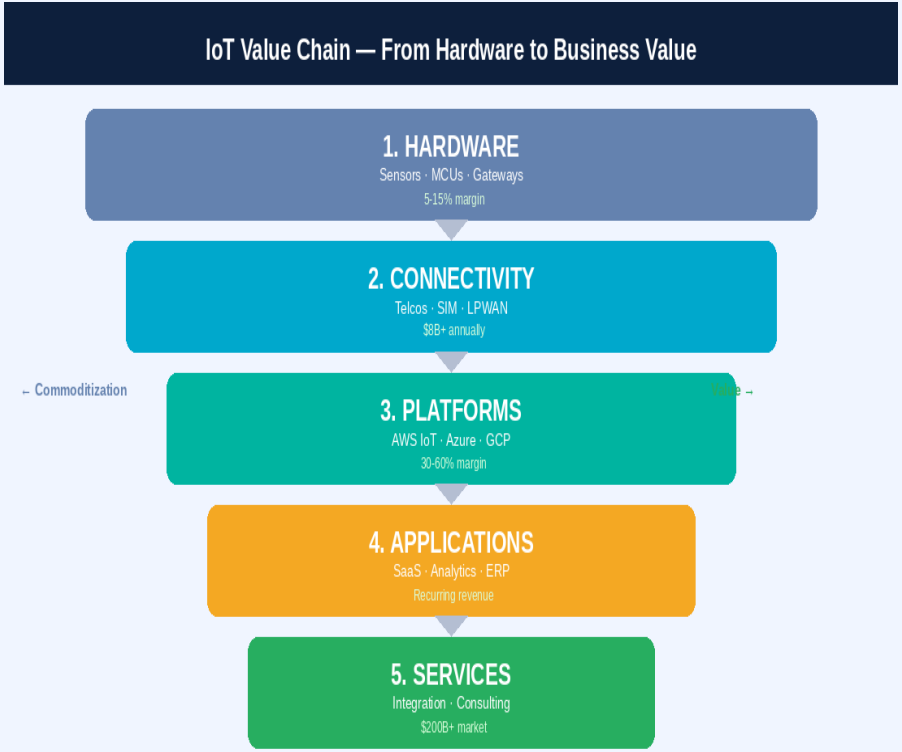
1	Hardware & Devices	Margin: 5–15%	Physical components: sensors, MCUs, gateways, actuators. The commodity layer — hardware is a prerequisite but has the lowest margins because it can be copied and manufactured cheaply anywhere in the world.	*Bosch sensors, NXP chips, Raspberry Pi, Cisco gateways*
2	Connectivity	Margin: Stable	Network infrastructure that transmits data: cellular SIMs (NB-IoT, LTE-M), LoRa networks, Wi-Fi infrastructure. Telcos earn per-device monthly fees.	*Vodafone IoT SIM, Helium Network, AT&T IoT Plans*
3	IoT Platforms	Margin: 30–60%	Cloud software managing devices, ingesting data, applying rules, and providing APIs. The highest-growth, highest-margin segment. Switching costs are enormous — once built on a platform, migration is expensive.	*AWS IoT Core, Azure IoT Hub, Google Cloud IoT, PTC ThingWorx*
4	Applications & Analytics	Margin: SaaS RR	Vertical software applications built on top of platforms, delivering industry-specific insights. Recurring subscription revenue from domain expertise combined with IoT data.	*Samsara (fleet), Itron (utilities), Verkada (security)*
			System integrators who make everything work together in real	

★ Hardware commoditises → Connectivity stabilises → Platforms grow fast → Applications capture recurring value → Services

IoT Value Chain — From Hardware to Business Value



IoT Value Chain — From Hardware to Business Value



1 Hardware
Sensors, MCUs, gateways
Margins 5–15% — commoditising fast
Key players: Bosch, NXP, STMicro

2 Connectivity
Telcos, SIM/eSIM, LPWAN operators
$8B+ annual IoT SIM revenue
Key players: Vodafone, Verizon

3 Platforms
IoT cloud & device management
30–60% gross margins — highest value
Key players: AWS, Azure, GCP

4 Applications
Vertical SaaS & analytics dashboards
Recurring subscription revenue
Key players: Samsara, Itron, SAP IoT

5 Services
System integration & managed services
$200B+ market for integration expertise
Key players: Accenture, IBM, Deloitte

IoT Business Models — How Value is Monetized

Product-Centric Models

- Hardware sales: Sell IoT devices (cameras, sensors, meters). Low margin, competitive. Razor-blade: cheap device, premium subscription (Nest thermostat model).
- Freemium platform: Free basic IoT platform, charge for advanced features, analytics, scale. AWS IoT free tier → paid as usage grows.
- Service contracts: Sell 'peace of mind' — hardware + monitoring + maintenance bundled. Xerox's 'pay per page' model adapted to IoT.
- White-label: Build IoT platform and white-label to enterprises who sell as their own product. Common in retail and healthcare IoT.

Data-Centric Models

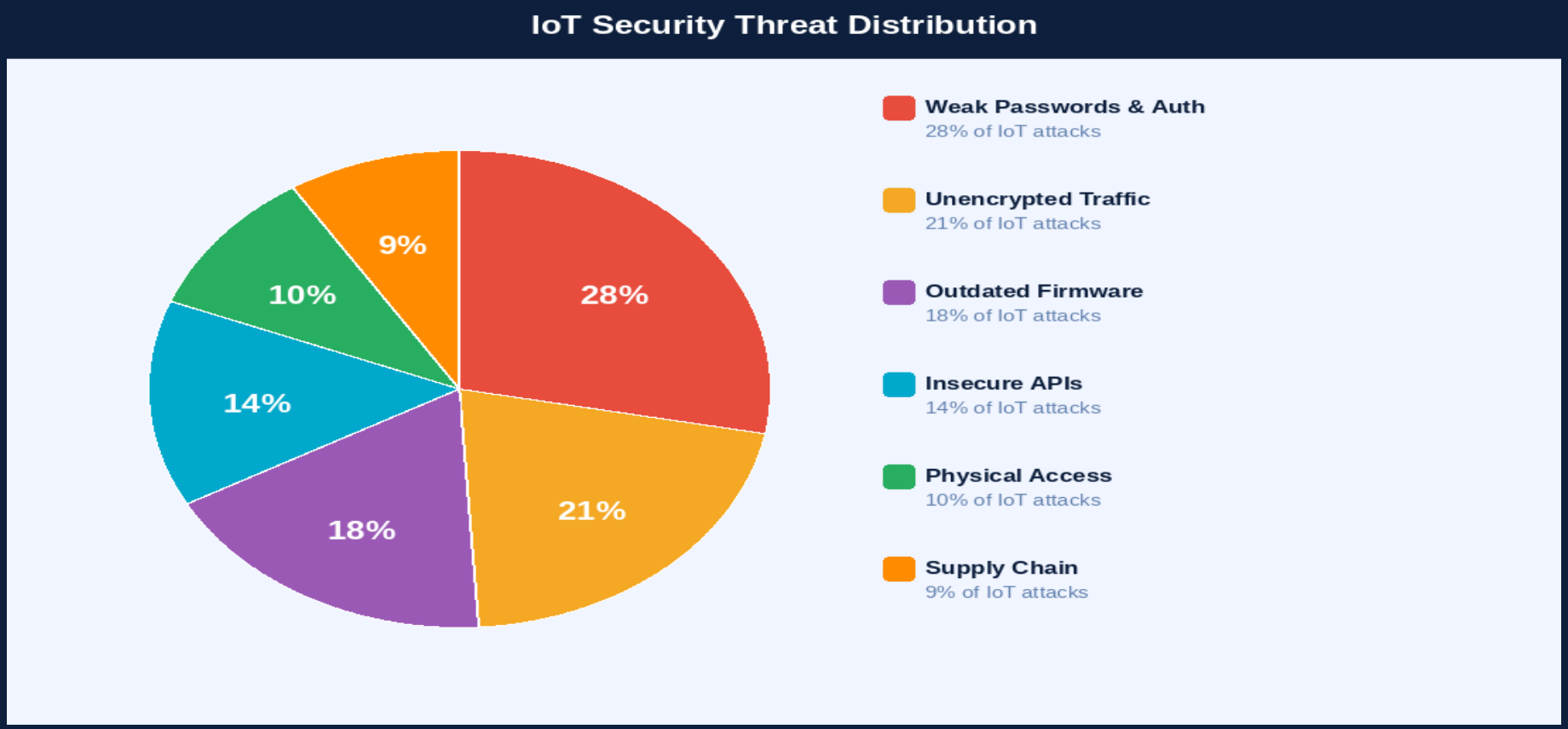
- Data monetization: IoT data sold to third parties (anonymized). Insurance companies buy driving behavior data from connected cars.
- Outcome-based pricing: Pay only for results — not hardware or software. 'Pay per landing' aircraft engine model (Rolls-Royce). 'Pay per ton crushed' mining equipment.
- Platform as a Service (PaaS): Charge developers to build IoT apps on your platform. AWS IoT, Azure IoT Hub — per message, per device per month.
- Subscription analytics: Monthly recurring revenue for insights dashboards, predictive alerts, compliance reports from IoT data streams.

Security, Privacy & Smart Applications

Threats · Defenses · Privacy · Governance · Smart Cities

- OWASP IoT Top 10
- Attack Case Studies
- Security Architecture
- Privacy Principles
- GDPR & Governance
- Smart City Use Cases

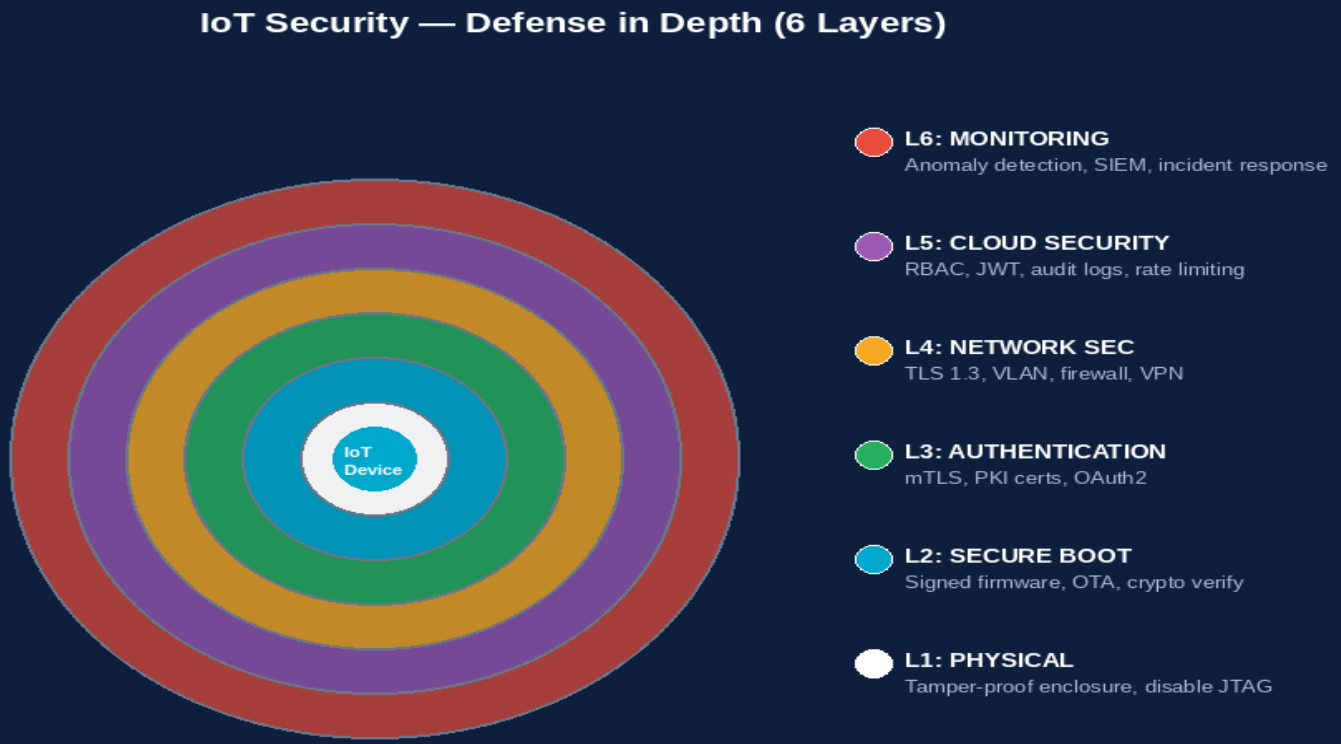
IoT Security Threat Distribution — Pie Chart



OWASP IoT Top 10 Security Risks

#1	**Weak/Hardcoded Passwords**	Default credentials never changed. Mirai botnet 600K devices → 1.2 Tbps DDoS.
#2	**Insecure Network Services**	Unnecessary open ports (Telnet 23, FTP 21). Disable all unused services.
#3	**Insecure Ecosystem Interfaces**	Web UI/API/mobile vulnerabilities. Broken auth, SQL injection. HTTPS + validation.
#4	**Lack of Secure Update Mechanism**	No OTA update — CVEs stay for years. Need signed, verified OTA with rollback.
#5	**Outdated/Vulnerable Components**	Deprecated OS, libraries with CVEs. IoT runs unsupported software 5-10 years.
#6	**Insufficient Privacy Protection**	Personal data without consent. GDPR violations up to 4% global revenue fine.
#7	**Insecure Data Transfer & Storage**	Plaintext MQTT/HTTP. Passwords on device flash. Use TLS in transit, AES at rest.
#8	**Lack of Physical Hardening**	JTAG/UART debug ports open. Attacker dumps firmware. Disable debug ports in production.
#9	**Insecure Default Settings**	FTP/Bluetooth/debug enabled by default. Principle of Least Privilege — disable all unneeded.
#10	**Lack of Monitoring & Logging**	No audit logs. Cannot detect intrusions. Implement syslog + SIEM integration.

IoT Security — Defense-in-Depth Visual



IoT Privacy — Concerns & Principles

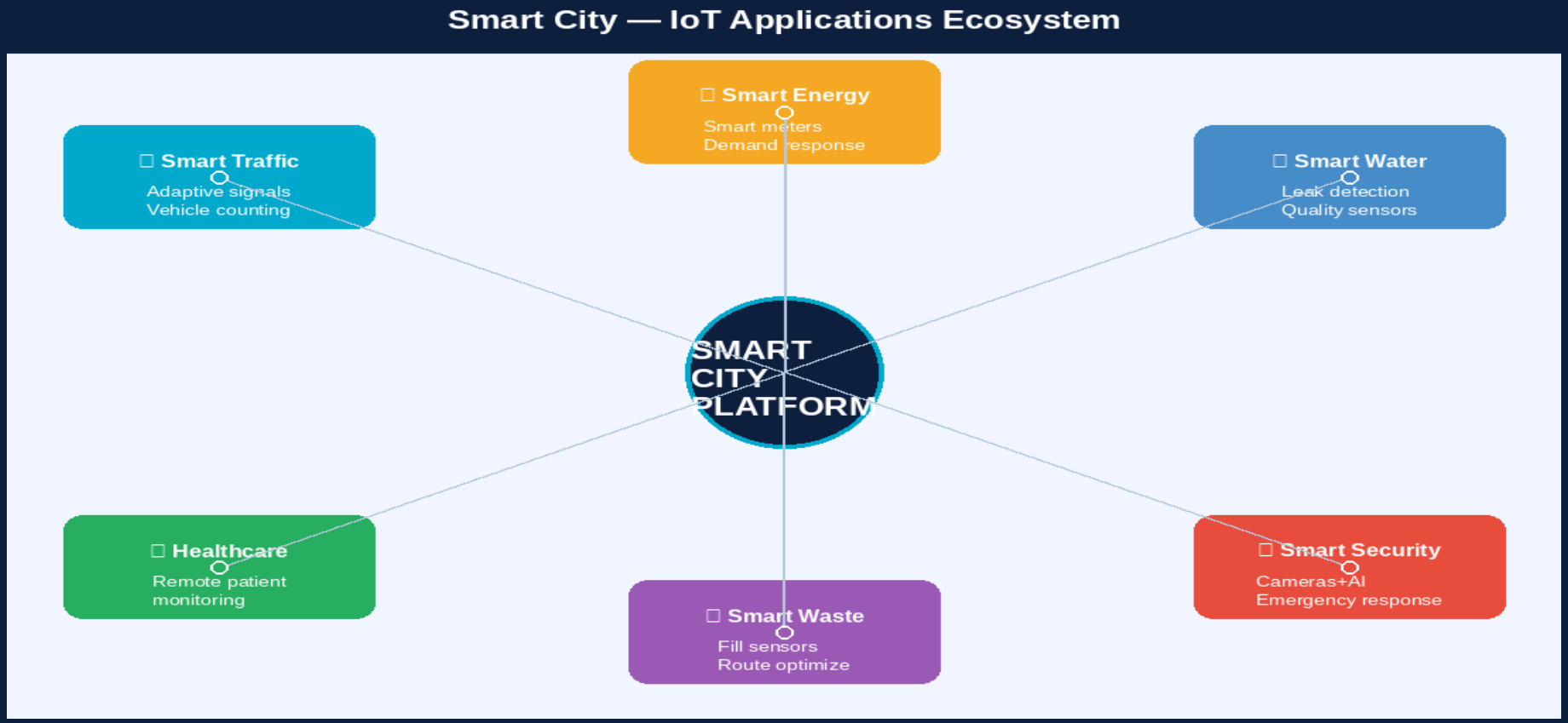
Privacy Risks in IoT

- Pervasive collection: Smart TVs track watching habits every 250ms. Smart speakers record all audio near activation word.
- Data aggregation: Smart home reveals when you wake, leave, who visits. Car reveals where you drive, what stores you park at.
- Third-party sharing: Health wearable data sold to insurers → premium hikes. Location data sold without user knowledge.
- Re-identification: 4 GPS points identify 95% of individuals uniquely (MIT study). Anonymization often fails in practice.
- Surveillance risks: Smart cameras, microphones enable mass surveillance by corporations or governments.
- Insecure storage: Personal health/location data stored unencrypted on cloud servers vulnerable to breach.

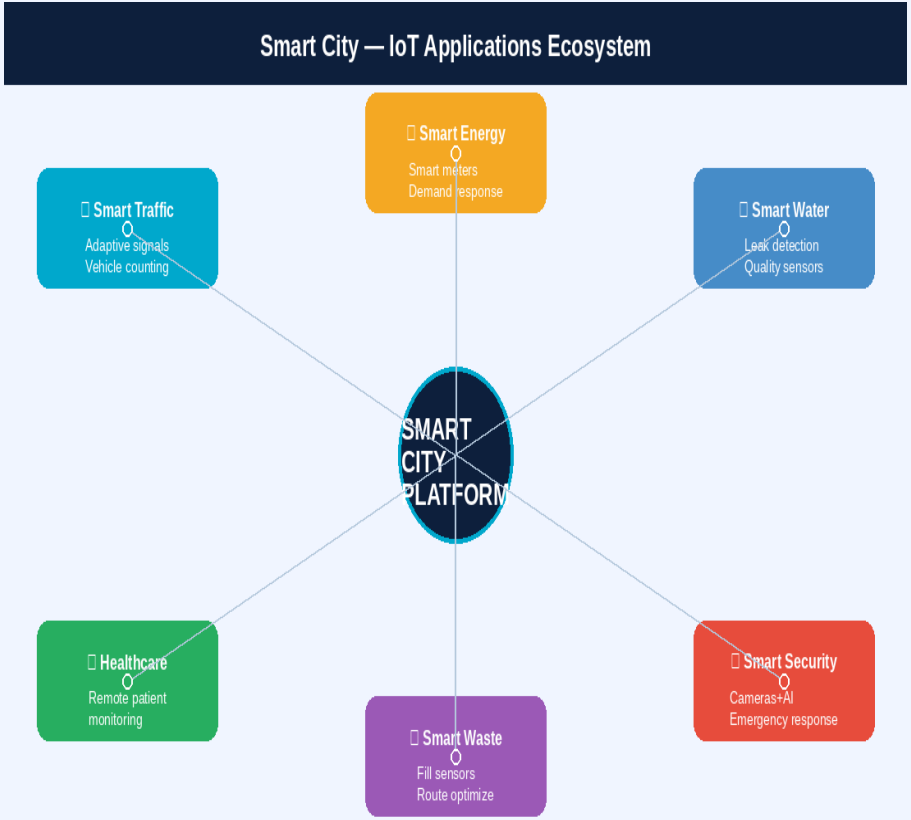
Privacy Principles & Solutions

- Data minimization: Collect ONLY what is strictly necessary for the stated purpose. Smart thermostat needs temp, NOT location history.
- Purpose limitation: Data collected for heating optimization CANNOT be sold to advertisers. Enforce contractually.
- Explicit consent: Users must actively agree (no pre-ticked boxes). Explain exactly what is collected and why.
- Right of erasure: Users can request all their data deleted. IoT platforms must support full data deletion API.
- Storage limitation: Define retention periods. Delete data when no longer needed. Typical: 90-day sensor data retention.
- Privacy by design: Build privacy into IoT system architecture from start — not bolted on afterward.

Smart City — IoT Applications



Smart City



Smart Traffic

Adaptive signals (SCOOT)
20–30% commute reduction
Emergency vehicle preemption

Smart Energy

AMI smart meters (2-way)
Demand response to grid
Predictive fault detection

Smart Water

Leak detection < 5 minutes
Quality: pH, chlorine, turbidity
Zone consumption analytics

Smart Waste

Fill-level ultrasonic sensors
40% fewer collection runs
Route optimisation per day

Smart Lighting

Motion-activated dimming
50–70% energy savings
Auto fault reporting + EV charge

Smart Security

AI video analytics
Crowd density detection
Emergency dispatch integration

Other IoT Applications —

Healthcare (IoMT)

- Remote Patient Monitoring (RPM): wearables measure ECG, SpO2, blood glucose continuously
- 40% reduction in hospital readmissions — problems caught at home before they escalate
- Smart hospital asset tracking: RFID on crash carts, IV pumps, staff locations in real-time
- Ingestible smart pills with sensors report GI tract data from inside the patient
- AI-driven sepsis early warning: ICU sensor streams trigger alert 6 hours before onset
- Standard: HL7 FHIR for health IoT data interoperability with EHR systems

Industrial IoT (IIoT / Industry 4.0)

- Predictive maintenance: vibration + thermal sensors detect bearing failure 3 weeks early
- 70% reduction in unplanned downtime — machines alert before they break
- Digital twins: virtual replica of factory machine updated in real-time from IoT sensors
- Collaborative robots (cobots) with force sensors work safely alongside humans
- Energy optimisation: machine-level energy sensors identify waste and reduce consumption 20%
- Standard: OPC-UA for factory floor sensor interoperability with SCADA and ERP

Smart Agriculture (Precision Farming)

- Soil moisture + weather forecast → irrigate only when/where needed: 40% water saving
- Multispectral drone imaging identifies crop stress zones invisible to human eye
- Livestock tracking: GPS + health sensors on cattle detect illness and location 24/7
- Micro-climate sensors in greenhouses maintain optimal temperature & CO2 automatically
- Yield prediction models combine satellite imagery + soil sensor data + historical harvest
- LoRaWAN is the dominant connectivity — 15km range, 10-year battery, no subscription